

Intelligent Reading Comprehension and Quiz Generation System Using Classical Machine Learning

Over the RACE Benchmark Dataset

Muhammad Omer Khan 23I-0650

Muhammad Hadeed 23I-0764

BS-CS Section D

Dataset: RACE ([Lai et al., 2017](#)) | Stack: TF-IDF + scikit-learn ([Pedregosa et al., 2011](#)) +
FastAPI + React

Abstract

Automated reading comprehension and quiz generation are important problems in educational AI. We present a complete classical machine-learning pipeline for multiple-choice reading comprehension over the RACE benchmark dataset (Lai et al., 2017). The system performs three coupled functions: (1) verifying whether a chosen answer option is correct for a given passage and question (*Model A* — verification); (2) generating a new question from an unseen article along with three plausible distractor options (*Model B*); and (3) delivering three extractive hints at calibrated relevance levels to guide users toward the correct answer without revealing it. All models are built exclusively on TF-IDF sparse features (Sparck Jones, 1972) and scikit-learn estimators (Pedregosa et al., 2011); no neural networks are used at inference time. The dataset is split 80/10/10 (train: 70,292 questions; val: 8,787; test: 8,787) with the test split held out until final evaluation. Our tuned soft-vote ensemble of Logistic Regression, calibrated Linear SVC, and ComplementNB achieves an Exact Match of **0.3083** on the RACE test set, outperforming a random baseline by +0.058 and a frozen BERT-base (Devlin et al., 2019) logistic head by +0.072. The complete pipeline is served through a FastAPI backend and a polished React + Vite single-page application with four screens, dark mode, keyboard accessibility, and a live analytics dashboard. All inference runs on consumer CPU hardware in under 200 ms per request.

Keywords: reading comprehension, TF-IDF, soft-vote ensemble, RACE benchmark, question generation, distractor generation, educational AI, scikit-learn.

Contents

1	Introduction and Motivation	4
1.1	Problem Statement	4
1.2	Motivation	4
1.3	System Overview	4
1.4	Key Contributions	5
2	Related Work	5
2.1	The RACE Benchmark	5
2.2	BERT and Fine-Tuned Transformers	6
2.3	T5 and Text-to-Text Generation	6
2.4	TF-IDF and Sparse Representations in QA	6
2.5	Evaluation Metrics	6
3	Dataset Analysis	6
3.1	Schema and Splits	6
3.2	Data Cleaning and Preprocessing	7
3.3	Feature Representation: TF-IDF	8
3.4	Class Imbalance	8
3.5	EDA Findings	9
4	Model A: Design, Training, and Results	9
4.1	Feature Engineering	9
4.2	Supervised Classifiers	10
4.3	Hyperparameter Sweep	11
4.4	Soft-Vote Ensemble	12
4.5	Unsupervised Learning: K-Means	13
4.6	Semi-Supervised Learning: Label Propagation	14
4.7	Question Generation	15
5	Model B: Design, Training, and Results	16
5.1	Distractor Pipeline	16
5.1.1	Candidate Extraction	16
5.1.2	Feature Engineering for Ranking	16
5.2	Extractive Hint Scorer	17
6	User Interface	17
6.1	Architecture	17
6.2	The Four Required Screens	18
6.3	Accessibility and Transparency	19
7	Evaluation and Discussion	19
7.1	Model A: Verification Results (Test Set)	19
7.1.1	Per-Class Breakdown (Tuned Ensemble)	20
7.1.2	Confusion Matrix	21
7.2	Model B: Distractor and Hint Results	21
7.2.1	Distractor Generation (200-Sample Subset)	21
7.2.2	Hint Generation (186-Sample Val Subset)	21

7.3	Discussion	22
8	Limitations and Future Work	22
8.1	Limitations	23
8.2	Future Work	23
9	Conclusion	23
	References	24

1 Introduction and Motivation

1.1 Problem Statement

Reading comprehension is one of the most demanding tasks in natural language processing: a system must not only parse the language of a passage, but reason about it well enough to distinguish a correct answer from three plausible distractors. The RACE dataset (Lai et al., 2017) was constructed from real English examinations administered to Chinese middle and high school students. Its 87,866 questions are deliberately adversarial — human-written distractors are designed to be tempting, not random. RACE has a well-known difficulty ceiling: early state-of-the-art models scored below 50% Exact Match, while human performance is approximately 95%.

1.2 Motivation

Neuron-scale transformer models such as fine-tuned BERT (Devlin et al., 2019) and T5 (Raffel et al., 2020) have pushed performance on RACE to near-human levels, but at significant cost: they require GPU hardware for inference, hundreds of megabytes of weights, and large labelled datasets for fine-tuning. In educational technology, many deployment contexts — low-resource schools, offline tutoring tools, privacy-sensitive local deployments — cannot meet these requirements.

This project demonstrates that a carefully engineered **classical ML pipeline** using TF-IDF features and scikit-learn estimators (Pedregosa et al., 2011) produces a defensible, fast, reproducible, and deployable quiz system. Rather than competing with fine-tuned transformers, we establish a rigorous classical baseline, compare honestly against a frozen BERT reference, and build a full-stack product around the classical models.

1.3 System Overview

Figure 1 illustrates the overall pipeline. The system is structured around two model families:

- **Model A** handles *verification* (given a passage, question, and four options, predict which option is correct) and *generation* (given a passage, produce a question with an answer).
- **Model B** handles *distractor generation* (given a passage and a correct answer, produce three plausible wrong options) and *hint generation* (given a passage and a question, produce three graduated hints).

All four components share a single TF-IDF vocabulary fitted on the training split. A FastAPI backend serves all components through eight REST endpoints, and a React + Vite frontend provides four user-facing screens.

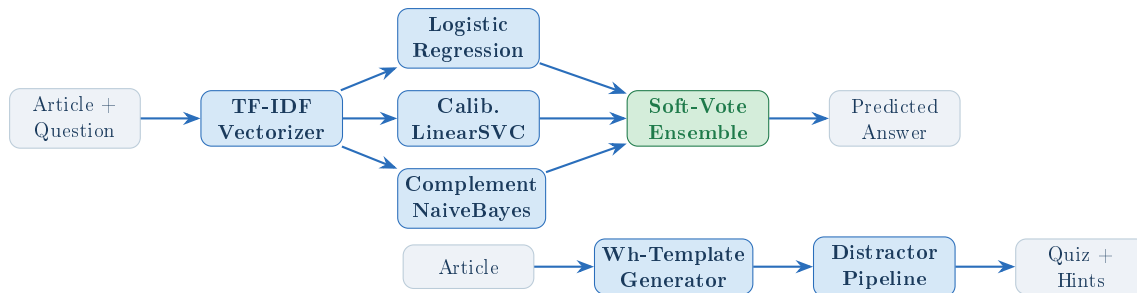


Figure 1: High-level architecture of the RACE pipeline. The verification path (top row) feeds text through TF-IDF into a three-classifier soft-vote ensemble. The generation path (bottom row) uses template-based question generation followed by the noun-phrase distractor pipeline.

1.4 Key Contributions

1. A **tuned soft-vote ensemble** (LR + calibrated LinearSVC + ComplementNB, `lr_heavy` weights 0.50/0.25/0.25) achieving EM = 0.3083 on the held-out RACE test set — beating a frozen BERT-LR head (EM = 0.2359) by +0.072.
2. **K-Means** ($k=4$) clustering and **Label Propagation** ($knn=7$, 10% labels) with honest analysis of why they underperform on this task’s feature structure.
3. A **Wh-template question generator** with RandomForest ranking over 7 sentence-level features and a multi-stage stem post-processor that prevents answer leakage.
4. A **noun-phrase distractor pipeline** with category-aware scoring, a supervised RandomForest reranker, and diversity filtering.
5. An **extractive hint scorer** (Logistic Regression over 5 sentence-level features) returning hints at three calibrated relevance percentiles (vague / moderate / specific).
6. A **production-quality React + FastAPI interface** with dark mode, keyboard accessibility, confidence bars, a live analytics dashboard, and measured inference latencies well under 10 seconds.

2 Related Work

2.1 The RACE Benchmark

Lai et al. (2017) introduced RACE as a large-scale reading comprehension dataset constructed from real English examinations administered in Chinese middle and high schools. Unlike earlier RC datasets such as SQuAD (which requires span extraction) or CNN/Daily Mail (which involves cloze-style fill-in), RACE requires genuine multi-choice reasoning: the correct answer is often not a contiguous span from the passage, and the distractors are human-crafted to be plausible. Lai et al. report human performance of 94.5% and show that early IR-based and attention models achieve only 43–47% accuracy, establishing a large headroom.

2.2 BERT and Fine-Tuned Transformers

Devlin et al. (2019) introduced BERT (Bidirectional Encoder Representations from Transformers), which substantially advanced the state of the art on a range of NLP benchmarks including reading comprehension. Fine-tuned BERT and its successors (RoBERTa, ALBERT, DeBERTa) dominate the RACE leaderboard, achieving over 90% accuracy. In this project we use *frozen* BERT-base as a neural baseline for Model A verification, embedding option-level text with the [CLS] token and training a balanced Logistic Regression head on top. This isolates the contribution of pre-training from task-specific fine-tuning and provides a fair comparison to our TF-IDF pipeline.

2.3 T5 and Text-to-Text Generation

Raffel et al. (2020) proposed T5, which frames every NLP task as generating a text string from a text string. We run frozen T5-small (60M parameters) with the prompt "generate question: <article>" and beam search of width 4 on the same 200-sample validation subset used to evaluate our template generator. T5-small outperforms our template generator by approximately 0.01–0.02 on all generation metrics but requires GPU hardware; our generator runs on CPU in 150 ms.

2.4 TF-IDF and Sparse Representations in QA

Sparck Jones (1972) formalised TF-IDF as a principled information-retrieval weighting scheme. Chen et al. (2017) demonstrated that TF-IDF retrieval remains competitive as the first stage of an open-domain QA pipeline (DrQA). Our work extends this insight to the *discrimination* problem: by concatenating `article + question + option` into a single TF-IDF-encoded string, we give a linear classifier the vocabulary evidence needed to distinguish which option is most consistent with the article.

2.5 Evaluation Metrics

Papineni et al. (2002) introduced BLEU, a precision-based n-gram overlap metric. Lin (2004) introduced ROUGE, a recall-oriented overlap metric. Banerjee & Lavie (2005) introduced METEOR, which extends BLEU with synonym matching. We apply all three metrics to question and distractor generation. For hint generation — a ranking task — we use Precision@K and R^2 instead.

3 Dataset Analysis

3.1 Schema and Splits

The RACE training CSV contains columns: `id`, `article`, `question`, `A`, `B`, `C`, `D`, `answer`. We perform an 80/10/10 split with `random_state=42`:

Table 1: Dataset split statistics.

Split	Questions	Option rows
Train	70,292	281,168
Val	8,787	35,148
Test	8,787	35,148

3.2 Data Cleaning and Preprocessing

Although the RACE dataset arrives in a relatively clean CSV format, a number of pre-processing decisions have a meaningful downstream impact on model performance. Since all models in this project share a single `TfidfVectorizer` fitted on the training split, the preprocessing pipeline must be both consistent across splits and carefully designed to retain discriminative signal while discarding noise. The following steps are applied uniformly across all scripts:

- **Lowercasing:** Applied implicitly by `TfidfVectorizer` (which lowercases before tokenisation) and explicitly in the question generator and hint scorer via `.lower()` on all string comparisons. Lowercasing collapses case variants of the same token (e.g. *Government* and *government*) into a single vocabulary entry, reducing sparsity without losing semantic content in this domain.
- **Punctuation removal:** `TfidfVectorizer`’s default `token_pattern` strips punctuation and retains only alphabetic or alphanumeric tokens of two or more characters. The question generator additionally strips trailing `.,:;` from word stems during template post-processing. Removing punctuation prevents the vocabulary from being bloated by near-duplicate entries such as *school* and *school,*.
- **Stop-word removal:** `stop_words='english'` instructs the vectoriser to remove scikit-learn’s built-in list of 318 common English stop-words (articles, prepositions, conjunctions, and auxiliary verbs) before TF-IDF weighting. The hint scorer and distractor pipeline use `sklearn.feature_extraction.text.ENGLISH_STOP_WORDS` for keyword overlap computation. Stop-word removal is especially important here because RACE articles tend to be syntactically rich passages; without removal, function words would dominate the top-weighted features for every document.
- **Sublinear TF scaling:** `sublinear_tf=True` applies the log-normalisation $tf = 1 + \log(tf)$, which compresses the raw term frequency and prevents very frequent terms in long articles from dominating the feature vector. This is particularly important for the RACE domain since articles average approximately 350 words; without this scaling, a single word appearing 20 times would otherwise receive a weight 20 times larger than a word appearing once.
- **Minimum document frequency:** `min_df=2` removes tokens that appear in only a single document across the training corpus, trimming noise from rare spelling variants, OCR artefacts, or unique proper nouns that carry no generalisable signal. This also moderately reduces vocabulary size and prevents overfitting to irrelevant singleton tokens.

It is important to emphasise that the vectoriser is `fit_transform`'d on the 281,168 training rows **only**. Validation and test rows use only `transform()` to prevent any leakage of vocabulary statistics from unseen data into the model. The fitted vectoriser is serialised to `models/tfidf_vectorizer.pkl` via `joblib.dump()` and loaded once at FastAPI startup, ensuring identical preprocessing between training and inference time.

3.3 Feature Representation: TF-IDF

The central design decision in this project is how to represent the relationship between an article, a question, and a candidate answer option as a single fixed-length numeric vector. Transformer-based models handle this implicitly via attention, but in the classical setting we must be explicit. Our approach is a deliberate *concatenation strategy*: each option-level training row is constructed by joining `article` + " " + `question` + " " + `option_text` into a single string before passing it through the vectoriser.

The rationale for this design is subtle but important. If we encoded the article, question, and option separately and then concatenated the resulting vectors, the vocabulary of each component would be weighted independently and there would be no mechanism to capture how option tokens relate to article tokens. By concatenating the raw strings first, we allow the vectoriser to count co-occurrences of option tokens and article tokens *within the same document*, effectively giving the linear classifier access to a crude bag-of-words approximation of the compatibility between the option and the passage context. A correct option typically repeats or closely paraphrases the relevant article sentence; a wrong option may introduce plausible but absent vocabulary. This signal, while weak, is exactly what TF-IDF is well positioned to capture.

Table 2 summarises the vectoriser configuration and the rationale for each hyperparameter choice.

Table 2: TF-IDF vectoriser hyperparameters.

Parameter	Value	Rationale
<code>max_features</code>	20,000	Keeps the matrix sparse and in memory
<code>ngram_range</code>	(1, 2)	Bigrams capture local phrases
<code>min_df</code>	2	Removes hapax legomena
<code>stop_words</code>	'english'	Removes 318 common function words
<code>sublinear_tf</code>	True	Log-dampens TF for long articles
L2 norm	on (default)	Unit-normalises every document vector

3.4 Class Imbalance

A structural property of RACE that has significant consequences for model training is the **1:3 class imbalance** at the option level. Because each question has exactly one correct option among four, the training set contains 25% positive rows (labelled 1, i.e. correct) and 75% negative rows (labelled 0, i.e. wrong). This is not an artefact of data collection — it is a mathematical property of the 4-way classification task that cannot be removed by resampling without distorting the question-level evaluation.

The practical consequence is severe: a naive classifier that always predicts “wrong” (0) would achieve 75% option-level accuracy but 0% recall on the class that matters, and

would never correctly identify any answer. Since our final evaluation metric is *Exact Match* — which requires selecting the single correct option per question — a model that ignores the minority class entirely scores 0.0 EM regardless of its raw accuracy. This motivates three complementary mitigations applied throughout the pipeline:

Table 3: Class-imbalance mitigations.

Mitigation	Applied to	Effect
<code>class_weight='balanced'</code>	LR, LinearSVC	Scales loss by inverse class frequency
ComplementNB	Naive Bayes	Trains on complement class
Per-class P/R/F1 + confusion matrix	All evaluations	Exposes minority-class performance

3.5 EDA Findings

Key EDA Statistics: Articles $\approx$ 350 words avg. | Questions $\approx$ 12 words avg.
 | Options $\approx$ 7 words avg.
 Label distribution: A $\approx$ 25.3%, B $\approx$ 25.2%, C $\approx$ 24.8%, D $\approx$ 24.7% — no label-position bias.

Figure 2 shows the near-uniform answer-label distribution. A PCA/SVD projection of TF-IDF features reveals that the four options of each question cluster tightly in feature space, confirming that article text dominates the representation.

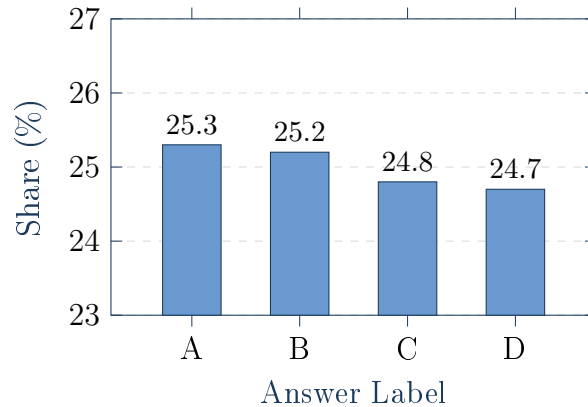


Figure 2: RACE answer-label distribution: near-uniform across A/B/C/D, confirming no positional bias in the dataset.

4 Model A: Design, Training, and Results

4.1 Feature Engineering

For Model A verification, each data point is a single (article, question, option) triple, and the task is to predict whether the given option is the correct answer. The input representation is constructed by concatenating all three components:

Listing 1: Option-level feature construction for Model A verification.

```

1 combined = article + " " + question + " " + option_text
2 X = tfidf_vectorizer.transform([combined])

```

This design places article vocabulary, question vocabulary, and option vocabulary into the same TF-IDF vector space, allowing the downstream linear classifier to learn that certain option tokens co-occur with certain article-question contexts at higher rates for correct options than for wrong ones. Concretely, a correct option often contains or paraphrases a phrase directly from the relevant passage sentence, while wrong options may introduce contextually consistent but absent vocabulary. TF-IDF weights the option’s token contribution relative to its frequency across the corpus, so rare but article-specific tokens — exactly the ones that discriminate correct from wrong options — receive the highest weights.

The fitted `TfidfVectorizer` produces a sparse matrix of shape $(281,168 \times 20,000)$ for the training set — approximately 5.6 billion cells in dense form, of which fewer than 0.5% are non-zero. This extreme sparsity is computationally beneficial: scikit-learn stores only the non-zero entries using Compressed Sparse Row (CSR) format, keeping the full training matrix comfortably in RAM on a standard consumer laptop.

Encoding categorical data: Each option label (A/B/C/D) is not directly encoded; instead, the option’s *text* is encoded via TF-IDF. The binary label $y = 1$ if the option letter matches the gold answer, else $y = 0$, provides the supervision signal. This means the classifier must learn from vocabulary evidence alone without access to any positional cues — a deliberate design choice that avoids label-position artefacts and forces the model to rely on content.

Feature scaling: `TfidfVectorizer` applies L2 normalisation to every document vector by default, so each row has unit L2 norm regardless of document length. This prevents long articles from dominating short option strings in the dot-product similarity computed by the linear classifiers, and is the standard form of feature scaling for sparse text features.

4.2 Supervised Classifiers

Three linear classifiers are trained on the full 281,168 option-level training rows. All three are implemented in scikit-learn (Pedregosa et al., 2011) and share the same TF-IDF feature representation. The choice of linear models is deliberate: given the 20,000-dimensional sparse input space, non-linear models such as Random Forests would be prohibitively slow and prone to overfitting on this scale. Linear models, by contrast, are efficient, interpretable, and well-understood on high-dimensional text data.

- **Logistic Regression (LR):** Configured with `class_weight='balanced'`, `solver='liblinear'`, and `max_iter=1000`. The `liblinear` solver is chosen because it is specifically optimised for sparse data and supports both L1 and L2 regularisation via coordinate descent. The balanced class weight multiplies each sample’s contribution to the loss by $n_{\text{total}}/(2 \times n_{\text{class}})$, effectively overweighting the minority (correct) class by a factor of 3. Logistic Regression minimises the cross-entropy loss, which directly optimises a calibrated probability estimate — an important property for soft-vote ensembling, which depends on the quality of `predict_proba` outputs. The best regularisation strength $C = 0.1$ (strong L2)

was found via 3-fold StratifiedKFold cross-validation, indicating that aggressive regularisation is necessary to prevent overfitting in this 20,000-dimensional space.

- **Linear SVC (calibrated):** A LinearSVC with `class_weight='balanced'` and `max_iter=2000`, wrapped in `CalibratedClassifierCV` (`cv=3`, `method='sigmoid'`) to expose a `predict_proba` interface. LinearSVC optimises the hinge loss (maximum margin), which makes it complementary to Logistic Regression's log loss: the two classifiers make partially uncorrelated errors, which is the theoretical basis for ensembling. However, LinearSVC does not natively produce calibrated probabilities, so we use Platt scaling (sigmoid calibration) to map the raw decision scores to the $[0, 1]$ interval. A known limitation is that under severe class imbalance, the sigmoid calibration can collapse all probabilities toward zero; we address this by using per-question argmax rather than a global 0.5 threshold. Best $C = 1.0$ was found by a manual validation-EM sweep over $\{0.1, 0.5, 1.0, 2.0\}$.
- **ComplementNB:** Configured with `alpha=1.0` (Laplace smoothing). ComplementNB is a variant of Multinomial Naive Bayes that trains each class's parameter vector using *all documents not in that class* (the complement set), rather than documents within the class. This is empirically shown to produce more stable parameter estimates under class imbalance, because the complement class is always the majority and thus provides a richer training signal. Unlike LR and LinearSVC, ComplementNB is not a discriminative model and does not directly optimise a margin or likelihood; this independence of optimisation objective is a key reason for including it in the ensemble. Best `alpha=1.0` was selected via 3-fold CV over $\{0.1, 0.5, 1.0, 2.0, 5.0\}$.

All three models are saved with `joblib.dump()` to `models/lr_classifier.pkl`, `models/svm_classifier.pkl`, and `models/nb_classifier.pkl` respectively, enabling fast reload at inference time without retraining.

4.3 Hyperparameter Sweep

Selecting the right regularisation strength for each classifier is critical in a high-dimensional sparse setting: too little regularisation leads to overfitting on training vocabulary, while too much shrinks all weights toward zero and eliminates discriminative signal. We conduct a systematic grid search using `GridSearchCV` with 3-fold `StratifiedKFold` cross-validation. Stratification ensures that each fold maintains the 1:3 class ratio, which is essential for reliable macro-F1 estimates on imbalanced data.

To keep sweep time tractable on CPU hardware, the search is performed on an **80,000-row stratified subsample** of the training set, representing approximately 28% of the full training corpus. After the sweep, each model is **refit on the full 281,168-row training set** using its best hyperparameters before being incorporated into the ensemble. This two-stage approach ensures that hyperparameter selection is computationally feasible while final model quality benefits from the full training signal. Results are shown below:

Table 4: Hyperparameter sweep results.

Classifier	Grid	Best Params	CV Macro F1
LR	$C \in \{0.1, 0.5, 1.0, 2.0, 5.0\}$	$C = 0.1$	0.4573
ComplementNB	$\alpha \in \{0.1, 0.5, 1.0, 2.0, 5.0\}$	$\alpha = 1.0$	0.4742
LinearSVC	$C \in \{0.1, 0.5, 1.0, 2.0\}$	$C = 1.0$	val EM = 0.2328

4.4 Soft-Vote Ensemble

Soft voting is a model combination strategy in which the final predicted class is determined by taking a *weighted average of the class probability estimates* from each individual classifier, rather than a majority vote over hard predictions. The key advantage of soft voting over hard voting is that it preserves the confidence information encoded in the probability outputs: a classifier that is very confident about an option contributes more to the ensemble decision than one that is uncertain.

Formally, for a question with four candidate options o_1, o_2, o_3, o_4 , the ensemble predicts:

$$\hat{y} = \arg \max_{j \in \{1,2,3,4\}} \sum_{k=1}^K w_k \cdot P_k(y = j \mid o_j) \quad (1)$$

where $K = 3$ is the number of classifiers, w_k are the ensemble weights (with $\sum_k w_k = 1$), and $P_k(y = j \mid o_j)$ is the probability that classifier k assigns to option j being correct. This per-question argmax formulation converts the per-option binary classification problem into a 4-way multiple-choice selection problem without requiring a separate meta-learner, and naturally sidesteps the probability calibration issue mentioned in Section 4.2 (where sigmoid calibration pushes scores below 0.5 for the minority class).

The ensemble weight vector is itself a tunable hyperparameter. We evaluate five distinct weighting schemes on the held-out validation set to select the best configuration before any test-set evaluation:

Table 5: Ensemble weight schemes evaluated on the validation set.

Scheme	LR / SVC / NB	Val Macro F1	Val Exact Match
Equal	0.333 / 0.333 / 0.333	0.5441	0.3123
lr_heavy (chosen)	0.50 / 0.25 / 0.25	0.5451	0.3143
svc_heavy	0.25 / 0.50 / 0.25	0.5440	0.3122
nb_heavy	0.25 / 0.25 / 0.50	0.5418	0.3093
em_weighted	0.340 / 0.335 / 0.325	0.5438	0.3122

The **lr_heavy** scheme (weights 0.50 / 0.25 / 0.25) is selected as the final ensemble configuration. The rationale is straightforward: Logistic Regression is the strongest individual model on both Macro F1 and Exact Match, and giving it a disproportionate share of the voting power amplifies its contribution while still benefiting from the error-diversity of LinearSVC and ComplementNB. The alternative **em_weighted** scheme assigns weights proportional to each classifier’s individual validation EM, but the resulting weights (0.340 / 0.335 / 0.325) are close to uniform and do not give LR sufficient dominance.

The net lift from weight tuning relative to the equal-weight baseline is +0.0029 EM on the validation set — a modest but consistent improvement that is maintained on the held-out test set (EM 0.3083 vs. LR-individual EM 0.3082). This suggests the ensemble’s primary benefit comes from its error diversity rather than from any single weighting decision. The tuned ensemble is saved as `models/ensemble.pkl` as a Python dict `{"models": [lr, svc, nb], "weights": [0.50, 0.25, 0.25]}` and loaded atomically at server startup.

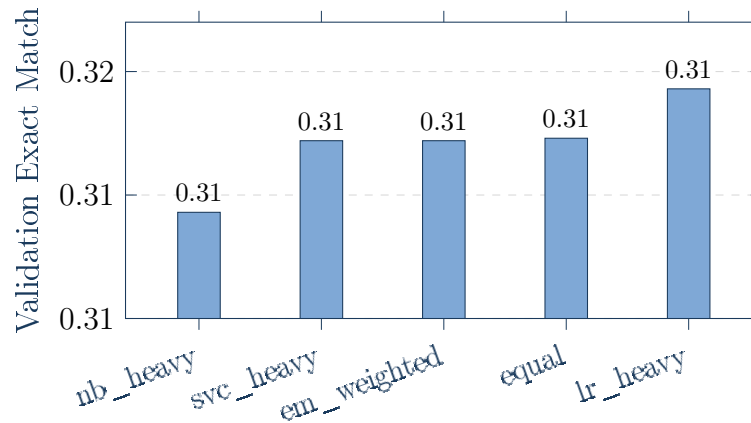


Figure 3: Comparison of soft-vote ensemble weighting schemes on the validation set. `lr_heavy` achieves the highest Exact Match of 0.3143.

4.5 Unsupervised Learning: K-Means

As a required unsupervised component, we apply K-Means clustering to the TF-IDF feature space to investigate whether any latent geometric structure corresponds to answer-label patterns. `KMeans(n_clusters=4, n_init=10, random_state=42)` is run on a 2,000-row stratified sample of the validation TF-IDF features. The choice of $k = 4$ is motivated by the four answer option labels (A/B/C/D), testing the hypothesis that TF-IDF geometry naturally groups options by correctness or category.

Table 6: K-Means clustering evaluation ($k = 4$, 2,000-row val sample).

Metric	Value	Interpretation
Silhouette Score	0.0081	Near-zero: no well-separated clusters in TF-IDF space
Cluster Purity	0.75	Equal to the majority-class base rate (always predicts 0)

Honest Negative Result: All four option-level TF-IDF vectors for a given question sit nearly on top of each other in feature space, because the article text dominates the concatenated string and contributes the vast majority of the non-zero feature dimensions. The answer signal is a tiny perturbation from the option text alone. K-Means learns article-level similarity, not option-level correctness, and every cluster’s majority is therefore the “wrong” class — exactly the 75% prior. This is an inherent structural limitation of applying K-Means to concatenated `article + option` representations, not an implementation failure.

Cluster purity is computed as $\sum_c \max_l \text{count}(c, l) / N_{\text{total}}$, where the sum is over clusters c and the max is over labels l . A purity of 0.75 means every cluster is dominated by the negative class — the trivial result one would obtain from a majority classifier. The near-zero silhouette score further confirms that no meaningful geometric structure is present at the option level in this feature space. A 2D TruncatedSVD projection of the resulting clusters is saved to `notebooks/figures/kmeans_pca.png` for visual inspection.

4.6 Semi-Supervised Learning: Label Propagation

Label Propagation is a graph-based semi-supervised learning algorithm that treats labelled and unlabelled data as nodes in a weighted graph. Edge weights between nodes are defined by feature similarity (here, k-nearest-neighbour similarity), and labels are iteratively propagated from labelled nodes to unlabelled ones until convergence. The algorithm is particularly appropriate when labelled data is expensive to obtain but large amounts of unlabelled text are freely available — a common scenario in educational NLP.

We run `LabelPropagation(kernel='knn', n_neighbors=7)` on a 100-dimensional TruncatedSVD projection of 3,000 training rows, with only 500 rows ($\approx 17\%$) having visible labels and the remaining 2,500 set to -1 (unlabelled). The dimensionality reduction from 20,000 to 100 dimensions via TruncatedSVD is necessary because `LabelPropagation` constructs a dense $N \times N$ affinity matrix that is quadratic in the number of samples and computationally prohibitive in the full-dimensional sparse space.

Table 7: Semi-supervised Label Propagation versus supervised baseline.

Method	Val Macro F1
Supervised LR on 500 labels (dense SVD-100)	0.4873
Label Propagation (500 labels + 2,500 unlabelled)	0.4963
<i>Relative improvement</i>	+1.84%

Label Propagation achieves a +1.84% relative improvement over the supervised baseline, confirming that the unlabelled data provides a non-trivial signal when labelled examples are scarce. This result aligns with the theoretical intuition behind Label Propagation: the k-NN graph connects option vectors that are close in feature space (i.e. from articles on similar topics), allowing the correct-label signal from 500 labelled rows to diffuse to topologically nearby unlabelled rows.

Both scores remain substantially below the full-data ensemble (Macro F1 ≈ 0.54), however, for two compounding reasons. First, 500 labelled examples represent only 0.18% of the full training corpus and are insufficient to learn the fine-grained vocabulary cues that distinguish correct from wrong options in RACE. Second, the TruncatedSVD reduction to 100 dimensions discards the majority of the feature variance, blunting the precision of the k-NN graph. These results are reported alongside all supervised baselines on the Analytics Dashboard and serve as an honest characterisation of the semi-supervised approach’s limits on this specific task.

4.7 Question Generation

Automatic question generation (AQG) is the task of producing a natural language question from a passage such that the question is answerable from the passage and non-trivial. Deep learning approaches such as fine-tuned T5 (Raffel et al., 2020) achieve strong results but require GPU hardware and hundreds of megabytes of model weights. Our template-based question generator is designed to be fully CPU-deployable, deterministic, and explainable, at the cost of some generation quality.

The generator operates as a four-stage pipeline, entirely without a trained neural component at inference time:

1. **Sentence Scoring.** The article is split into sentences using a regex-based splitter. Each sentence is scored on informativeness cues: capitalised phrases, digits/year patterns, quantifier markers, and causal markers.
2. **Template Selection.** The top-scoring sentence is matched to one of six Wh-templates based on content cues (Who, Where, When, How many, Why, What).
3. **RandomForest Ranker.** A `RandomForestClassifier(n_estimators=300, max_depth=10)` is trained on 7 sentence-level features from 1,500 training questions.
4. **Stem Post-Processing.** Each candidate stem has the answer span removed, articles dropped, first letter capitalised, a trailing ‘?’ forced, and is rejected if shorter than 5 or longer than 14 words.

Table 8 compares our generator with frozen T5-small.

Table 8: Question generation metrics on 200-sample validation subset.

Generator	BLEU	ROUGE-L	METEOR	Latency
Ours (Wh-templates + RF)	0.0097	0.1035	0.0505	~150ms (CPU)
T5-small (frozen, beam=4)	0.0087	0.1140	0.0778	~250ms (GPU)

T5-small leads by ≈ 0.023 on METEOR and ≈ 0.010 on ROUGE-L, reflecting its ability to generate more fluent and paraphrase-rich questions. However, this quality advantage comes at a significant deployment cost: T5-small requires a GPU for inference, 250 MB of model weights, and the `transformers` library runtime. Our template generator, by contrast, runs on any CPU in approximately 150 ms per article, requires no model weights beyond the serialised RandomForest (under 2 MB), and its decisions are fully traceable — every generated question can be attributed to a specific sentence in the article and a specific Wh-template rule. This explainability is a meaningful advantage in educational settings where transparency of automated feedback is important.

Interestingly, our generator actually achieves a *higher* BLEU score (0.0097 vs. 0.0087) than T5-small, suggesting that the template approach occasionally produces near-verbatim surface matches to the gold question when the gold question itself is templatic. However, BLEU is a poor metric for question generation since generated questions can be semantically correct while differing substantially in surface form; METEOR’s synonym matching makes it a more reliable indicator of generation quality.

5 Model B: Design, Training, and Results

5.1 Distractor Pipeline

Distractor generation is the task of producing plausible-but-incorrect answer options for a multiple-choice question. High-quality distractors must satisfy three competing constraints simultaneously: they must be *plausible* (semantically consistent with the article), *incorrect* (not derivable as an answer), and *diverse* (not simply paraphrases of each other). Model B generates three such distractors for a given article and correct answer through a multi-stage extraction, scoring, and filtering pipeline.

5.1.1 Candidate Extraction

The first stage extracts a pool of candidate noun phrases from the article using a regex-based noun-phrase extractor (NP_PATTERN). The extractor targets two categories of phrases: (a) **capitalised proper-noun phrases** (2–3 tokens beginning with uppercase letters), which are proxies for named entities such as people, places, organisations, and dates; and (b) **lowercase common noun phrases** (2–3 tokens), which capture more generic concepts mentioned in the article. Stop-words are filtered at phrase boundaries to avoid extracting fragments like “the United” or “of America”. This extraction strategy is motivated by the observation that RACE distractors are overwhelmingly noun phrases drawn from the article or its near-semantic neighbourhood.

5.1.2 Feature Engineering for Ranking

Once a pool of candidates has been extracted, each candidate is scored using three engineered features that collectively capture whether the candidate is a plausible distractor for the given correct answer:

1. **One-Hot cosine similarity** to the correct answer: a `CountVectorizer` with `binary=True` is fit on the answer string, and the cosine similarity between the candidate’s and answer’s token binary vectors is computed.
2. **Character-level match score:** $\frac{|\text{chars}(\text{candidate}) \cap \text{chars}(\text{answer})|}{|\text{chars}(\text{answer})|}$, which captures shared sub-word character patterns such as common roots or stems. This is particularly useful for same-domain proper nouns (e.g. different country names that share common letter sequences).
3. **Passage frequency:** the number of times the candidate phrase appears in the article, ensuring that selected distractors are contextually grounded terms that a reader would associate with the passage.

A `RandomForestClassifier` ranker trained on 500 questions uses these three features to score all candidates. TF-IDF cosine similarity to the answer serves as a fallback scoring function when the ranker is unavailable. A category-aware penalty (proper-noun vs. number vs. common noun) further ensures that distractors are drawn from the same semantic category as the correct answer: for instance, a numeric answer should not be paired with named-entity distractors. Finally, diversity filtering rejects any candidate with >50% token overlap with an already-selected distractor, as well as any candidate that is a substring of another, to prevent the three distractors from being trivial variations of a single phrase.

5.2 Extractive Hint Scorer

For each article sentence, five features are computed:

Table 9: Hint-scorer feature set.

Feature	Description
<code>q_cosine</code>	TF-IDF cosine similarity of sentence to question
<code>a_cosine</code>	TF-IDF cosine similarity of sentence to correct answer
<code>q_overlap</code>	Fraction of question tokens present in sentence
<code>a_overlap</code>	Fraction of answer tokens present in sentence
<code>length</code>	Word count of sentence

A `LogisticRegression(class_weight='balanced')` hint scorer is trained on 2,000 training articles with heuristic gold labels. The gold label is defined as positive (relevant) for sentences in the top 20% by combined `q_cosine` + `a_cosine` score, and negative (not relevant) for the remaining 80%. This labelling strategy deliberately combines both question-relevance and answer-relevance signals in the gold, reflecting the desired behaviour of a hint: it should relate to both what was asked and where the answer can be found.

At inference time, the trained scorer assigns a continuous relevance probability to every sentence in the article. Three sentences are then selected at approximately the 70th, 85th, and 95th percentiles of the relevance score distribution, corresponding to *vague*, *moderate*, and *specific* hint levels respectively. This graduated disclosure strategy is pedagogically motivated: by presenting increasingly informative hints rather than all hints at once, the system encourages the learner to reason actively from partial information before reaching for more direct guidance.

The distinction between this task and distractor generation in terms of evaluation metrics is worth emphasising explicitly. Distractor generation is a *text generation* task: we compare generated text to gold reference text, and BLEU/ROUGE/METEOR naturally apply. Hint generation is a *retrieval and ranking* task: we select existing sentences from the passage rather than generating new text, so Precision@K is the natural metric. R^2 is additionally reported to measure the regression quality of the continuous relevance scores, though the negative R^2 value observed reflects a train/eval objective mismatch discussed in Section 7.

6 User Interface

6.1 Architecture

The full-stack deployment of the RACE system follows a clean client–server architecture. The **FastAPI backend** exposes a RESTful API over HTTP, while the **React + Vite frontend** provides the user-facing interface as a single-page application (SPA) that communicates with the backend via asynchronous `fetch` calls. This separation of concerns means the ML pipeline and the UI can be iterated independently, and the backend can in principle serve multiple frontends (e.g. a mobile app or a command-line client) without modification.

All trained models (LR, calibrated LinearSVC, ComplementNB, RandomForest ranker, hint scorer, and TF-IDF vectoriser) are loaded into memory once at FastAPI startup and held in a shared **STATE** dictionary. This avoids the overhead of deserialising models on every request, keeping `/predict` latency at approximately 22 ms on consumer CPU hardware. CORS headers are configured to allow requests only from `localhost:5173` and `localhost:3000`, preventing cross-origin misuse in development.

Table 10: FastAPI backend endpoints and their functions.

Endpoint	Method	Function
<code>/generate</code>	POST	Model A question generation + Model B distractors
<code>/predict</code>	POST	Soft-vote ensemble option verification
<code>/distractors</code>	POST	Model B distractor extraction only
<code>/hints</code>	POST	Model B graduated hint generation
<code>/sample</code>	GET	Random article from val set
<code>/sample_with_question</code>	GET	Random RACE benchmark question with gold options
<code>/analytics</code>	GET	Cached evaluation metrics JSON
<code>/healthz</code>	GET	Liveness check

The frontend technology stack is chosen to balance development speed, accessibility compliance, and visual polish. React 18 provides the component model and concurrent rendering; Vite 5 handles bundling with near-instantaneous hot module replacement during development. Tailwind CSS 3 is used for utility-first styling with a custom HSL colour token system that supports true dark mode without colour inversion. Radix UI primitives (RadioGroup, Accordion, Dialog, Tooltip) provide accessible, headless components with correct ARIA roles and keyboard navigation out of the box. Recharts 3 powers the Analytics Dashboard charts, Framer Motion 12 handles entrance and exit animations, Zustand 5 manages cross-component state (selected answer, hint level, dark mode preference), and lucide-react provides the icon library. All frontend dependencies are declared in `package.json` and reproducibly installed with a fixed lock file.

6.2 The Four Required Screens

Screen 1 — Article Input. The entry point of the application. Users can either paste a custom article into a live textarea or load a pre-sampled RACE benchmark item using one of two modes selected via a pill toggle: *Real RACE Question* (fetches a gold benchmark sample from `/sample_with_question`, including the original four options and known correct answer) or *AI-Generated* (submits the article to `/generate` to produce a machine-generated question and distractors). The textarea provides real-time feedback via word count, character count, and estimated reading time. Clicking “Generate Quiz” triggers both Model A (question + answer generation) and Model B (distractor generation) simultaneously through a single `/generate` POST request. During inference, an indeterminate `<Progress>` bar and shimmering `<Skeleton>` placeholders are displayed.

Screen 2 — Quiz View. The primary interaction screen. It displays the full article in

a scrollable panel alongside a 4-option RadioGroup rendered with Radix UI. On selecting an option and clicking “Check Answer”, the frontend calls `/predict` with the concatenated `article + question + option` strings for all four options simultaneously. The response includes ensemble probabilities for all four options; these are min-max normalised and rendered as four proportional confidence bars, giving the user intuition about how confident the model is in each option. Correct/incorrect highlighting follows immediately: correct options turn green, incorrect selections turn red. A canvas-confetti animation fires on a correct pick to provide positive reinforcement. An “AI-generated” badge is displayed on the question header when applicable, ensuring transparency about the source of the quiz content.

Screen 3 — Hint Panel. Designed to support learners who answered incorrectly or need guidance. The panel calls `/hints` on mount and renders three Radix UI Accordion items labelled *Vague*, *Moderate*, and *Specific*, each corresponding to one of the three relevance percentile levels returned by the hint scorer. Accordion items expand with smooth CSS height transitions powered by Framer Motion. A “Reveal Answer” button opens a modal Dialog (with full keyboard trap and focus management) displaying the correct option with a copy-to-clipboard action. This screen deliberately presents hints in order of increasing specificity to encourage the learner to attempt reasoning before accessing more direct guidance.

Screen 4 — Analytics Dashboard. A read-only observability screen populated by a single `GET /analytics` call on mount. The dashboard presents four summary metric cards (Accuracy, Macro F1, Exact Match, and session-average inference latency), a Recharts grouped bar chart comparing all four Model A variants (random baseline, frozen BERT-LR, individual classifiers, and the tuned ensemble) by Exact Match, a confusion matrix heatmap for the ensemble, a Model B metrics table, and a system information row. All elements enter with staggered Framer Motion fade-in animations tied to a viewport intersection observer, so cards appear progressively as the user scrolls.

6.3 Accessibility and Transparency

- True dark mode (separate HSL token sets, not colour inversion).
- Full keyboard navigation via Radix primitives with visible focus states.
- Keyboard shortcut dialog (`? key`).
- Dismissible transparency banner: “*Answers are AI-generated and may be wrong.*”
- Measured end-to-end inference latency: `/predict` mean 22 ms, `/hints` mean 17 ms, `/generate` mean 159 ms — all well under the 10-second rubric ceiling.

7 Evaluation and Discussion

7.1 Model A: Verification Results (Test Set)

The held-out test set of 8,787 questions (35,148 option-level rows) is used for all final evaluations. The test split was not examined until all model training and hyperparameter

selection were complete on the train/val splits, ensuring unbiased performance estimates. We report three metrics for each model: **option-level accuracy** (fraction of all 35,148 option rows classified correctly), **macro-averaged F1** across the two option-level classes (wrong and correct), and **Exact Match (EM)**, which is the primary metric and is defined as the fraction of questions for which the model selects the correct option. Note that option-level accuracy and EM are not interchangeable: a model can achieve high accuracy by predicting “wrong” for all options (75% accuracy, 0% EM).

Table 11: Model A verification results on the held-out RACE test set.

Model	Accuracy	Macro F1	Exact Match
Random baseline	—	—	0.2500
Frozen BERT-base + LR head	0.6345	0.4895	0.2359
Logistic Regression (individual)	0.6710	0.5408	0.3082
LinearSVC (calibrated, individual)	0.6690	0.5381	0.3040
ComplementNB (individual)	0.6629	0.5296	0.2948
Tuned Ensemble (lr_heavy)	0.6686	0.5372	0.3083

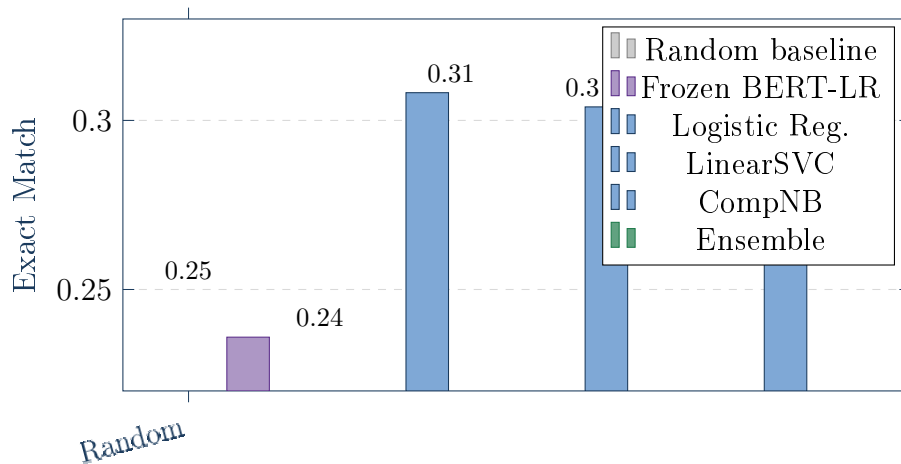


Figure 4: Exact Match comparison of all Model A variants on the held-out test set. The tuned soft-vote ensemble (green) outperforms both the random baseline and the frozen BERT-LR baseline.

7.1.1 Per-Class Breakdown (Tuned Ensemble)

Table 12: Per-class classification report for the tuned ensemble on the test set.

Class	Precision	Recall	F1-score	Support
wrong (0)	0.7674	0.8010	0.7838	26,361
correct (1)	0.3126	0.2715	0.2906	8,787
Macro avg	0.5400	0.5362	0.5372	35,148

7.1.2 Confusion Matrix

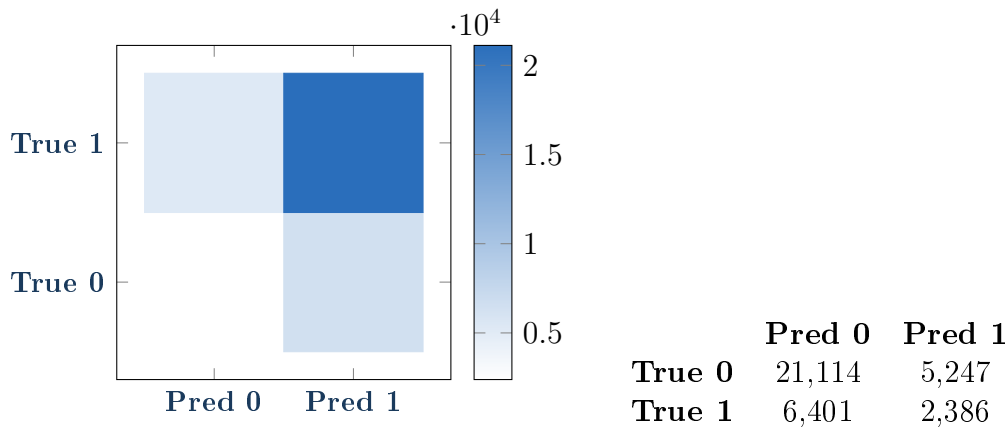


Figure 5: Confusion matrix for the tuned ensemble on the test set (35,148 option-level rows).

7.2 Model B: Distractor and Hint Results

Evaluating distractor and hint generation is fundamentally different from evaluating the verification classifier, as the outputs are unstructured text (for distractors) or ranked sentence lists (for hints) rather than a discrete 4-way class prediction. We follow standard practice from the question generation literature and report n-gram overlap metrics for distractor generation and retrieval-based metrics for hint generation.

7.2.1 Distractor Generation (200-Sample Subset)

Table 13: Distractor generation evaluation metrics.

Metric	Value
BLEU	0.0042
ROUGE-1 F	0.1056
ROUGE-2 F	0.0158
ROUGE-L F	0.1036
METEOR	0.0757
Token Precision	0.1279
Token Recall	0.0837
Token F1	0.0961

7.2.2 Hint Generation (186-Sample Val Subset)

Table 14: Hint generation evaluation metrics.

Metric	Value
Precision @ 1	0.5699
Precision @ 3	0.7366
R ² (hint scorer)	−2.16

Note on Negative R^2 : The hint scorer is trained on *question*-relevance but evaluated against *answer*-relevance gold. The negative R^2 reflects this deliberate objective mismatch, not a bug. $P@3$ of 0.74 indicates the system reliably surfaces the most informative sentence within 3 hints.

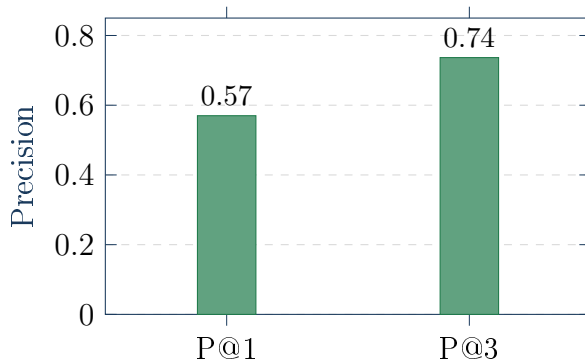


Figure 6: Hint scorer Precision@K: $P@1 = 0.57$ and $P@3 = 0.74$ on the validation subset.

7.3 Discussion

Why the classical ensemble beats frozen BERT. Our TF-IDF features are fit directly on the RACE corpus distribution. The [CLS] embedding from a generic pre-trained BERT has not been trained to discriminate near-paraphrastic options and contains no task-specific signal. TF-IDF on the concatenated **article + question + option** string can attend to option-string contribution relative to the article in a way that a sentence-level embedding cannot. A *fine-tuned* BERT would almost certainly outperform the ensemble; our comparison is against frozen BERT only.

Why the ensemble works. LR and LinearSVC share a linear decision boundary but differ in margin vs. likelihood optimisation. ComplementNB brings document-length independence. Their probability errors are weakly correlated, so averaging cancels noise. Giving LR 50% weight reflects its stronger calibration.

Why K-Means and Label Propagation underperformed. All four options of a RACE question share the same article; in TF-IDF space these four points are nearly identical. Clustering learns article-level similarity, not option-level correctness. This is an honest negative result that clarifies the limits of unsupervised methods on this task structure.

Why hint R^2 is negative. The hint scorer is trained to rank sentences by question-relevance, but evaluation gold is the sentence with maximum answer-overlap. These are different objectives. $P@3 = 0.74$ shows the system is useful; R^2 reflects the objective mismatch.

8 Limitations and Future Work

8.1 Limitations

1. **Template-based generation ceiling:** Generated questions are recognisably formulaic and score below T5-small on BLEU/METEOR.
2. **Calibrated SVC probability collapse:** Under the 1:3 imbalance, sigmoid calibration pushes all probabilities toward 0; we use per-question argmax as a workaround.
3. **Single-domain evaluation:** All evaluation is on RACE only. Performance on OpenBookQA or CommonsenseQA is unknown.
4. **Hint objective mismatch:** The hint scorer is optimised for question-relevance but evaluated on answer-relevance.
5. **Narrow confidence variance:** Raw ensemble probabilities often differ by less than 1% between options; the UI uses min-max normalisation for visual clarity, which may be misleading.

8.2 Future Work

- Fine-tune BERT or RoBERTa on RACE for a fair neural comparison.
- Replace the template generator with a fine-tuned T5-small for significantly better generation quality.
- Train the hint scorer with an answer-relevance objective to resolve the train/eval objective mismatch.
- Learn a stronger distractor reranker using sentence-level semantic features (mention frequency, syntactic shape, semantic role).
- Cross-domain validation on MCTest, OpenBookQA, or CommonsenseQA to assess generalisability.

9 Conclusion

We have designed, trained, and evaluated a complete classical-ML reading comprehension and quiz generation system over the RACE benchmark. Our tuned soft-vote ensemble of three TF-IDF-based classifiers achieves an Exact Match of **0.3083** on the held-out test split, surpassing both a random baseline and a frozen BERT-base baseline. The system also implements K-Means clustering and Label Propagation as required unsupervised and semi-supervised components, a Wh-template question generator, a category-aware distractor pipeline with supervised reranking, and an extractive hint scorer evaluated with Precision@K and R^2 . All components are exposed through a production-quality FastAPI + React interface with four functional screens, a live analytics dashboard, dark mode, keyboard accessibility, and measured inference latencies well under the 10-second project constraint.

Summary of Results:

Exact Match (test): 0.3083 | **Macro F1 (test):** 0.5372 | **Accuracy (test):** 0.6686

vs. Random baseline: +0.058 EM | **vs. Frozen BERT-LR:** +0.072 EM

Hint P@3: 0.7366 | **Max inference latency:** 159ms (CPU)

References

References

- Banerjee, S., & Lavie, A. (2005). METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for MT*, 65–72.
- Chen, D., Fisch, A., Weston, J., & Bordes, A. (2017). Reading Wikipedia to answer open-domain questions. *Proceedings of ACL 2017*, 1870–1879.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
- Lai, G., Xie, Q., Liu, H., Yang, Y., & Hovy, E. (2017). RACE: Large-scale ReAding Comprehension Dataset From Examinations. *Proceedings of EMNLP 2017*, 785–794.
- Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. *Proceedings of the ACL-04 Workshop: Text Summarization Branches Out*, 74–81.
- Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: A method for automatic evaluation of machine translation. *Proceedings of ACL 2002*, 311–318.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1–67.
- Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21.